

Generación Automática de Tests

Clase 13

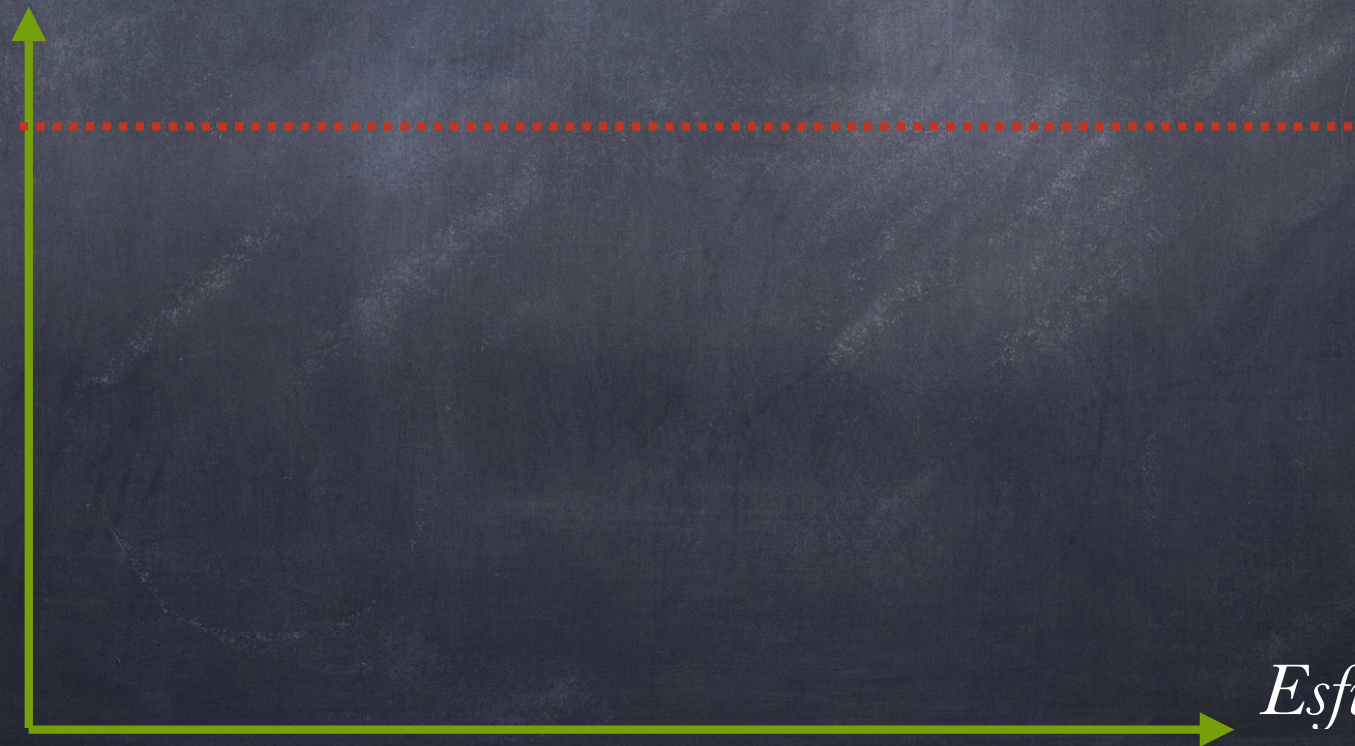
Generación Automática de Test: Generación Aleatoria guiada
por feedback

Valeria Bengolea

Enfoques

- Tradicional: No se cuenta con nada formal (excepto por el programa ejecutable).
- Formales: verificación deductiva, Model Checking

Garantía

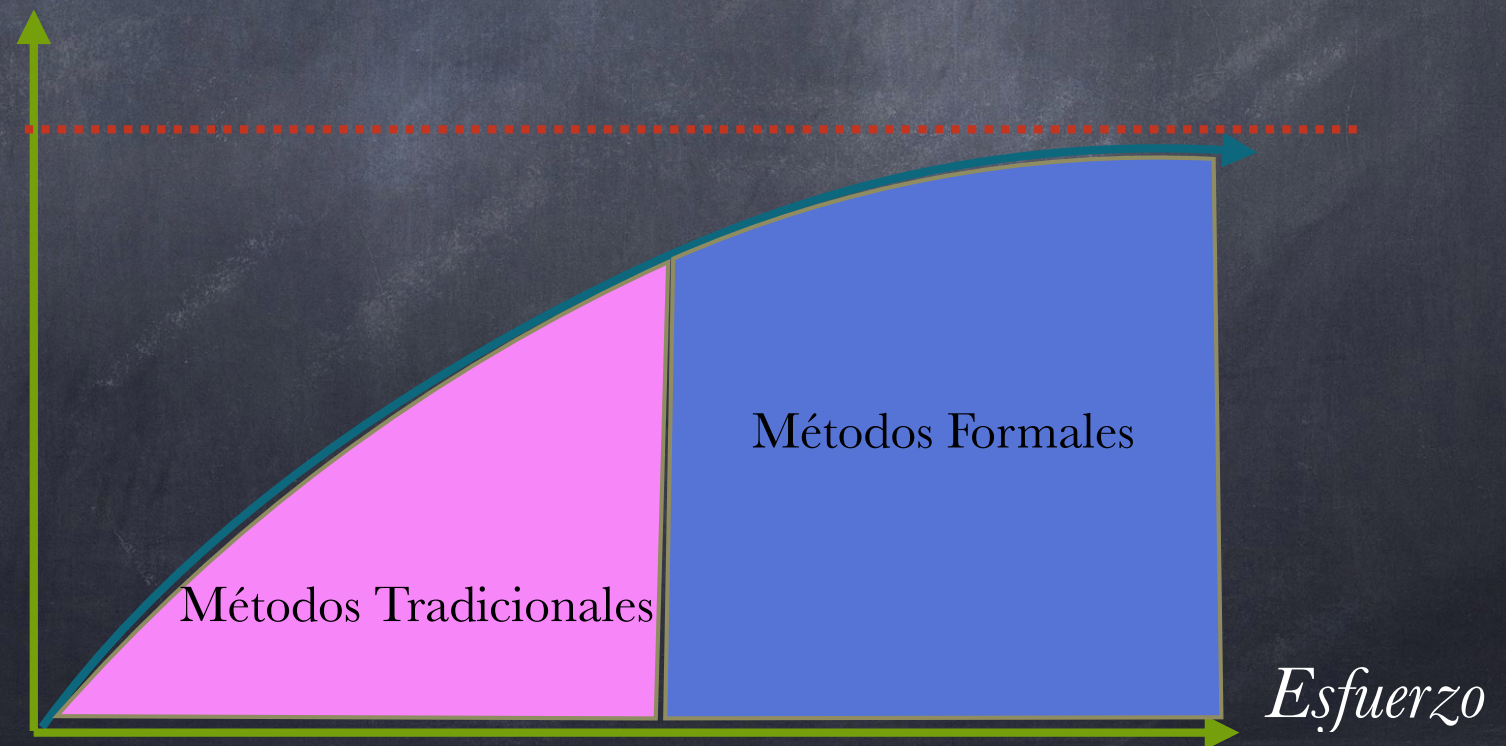


Esfuerzo

Enfoques

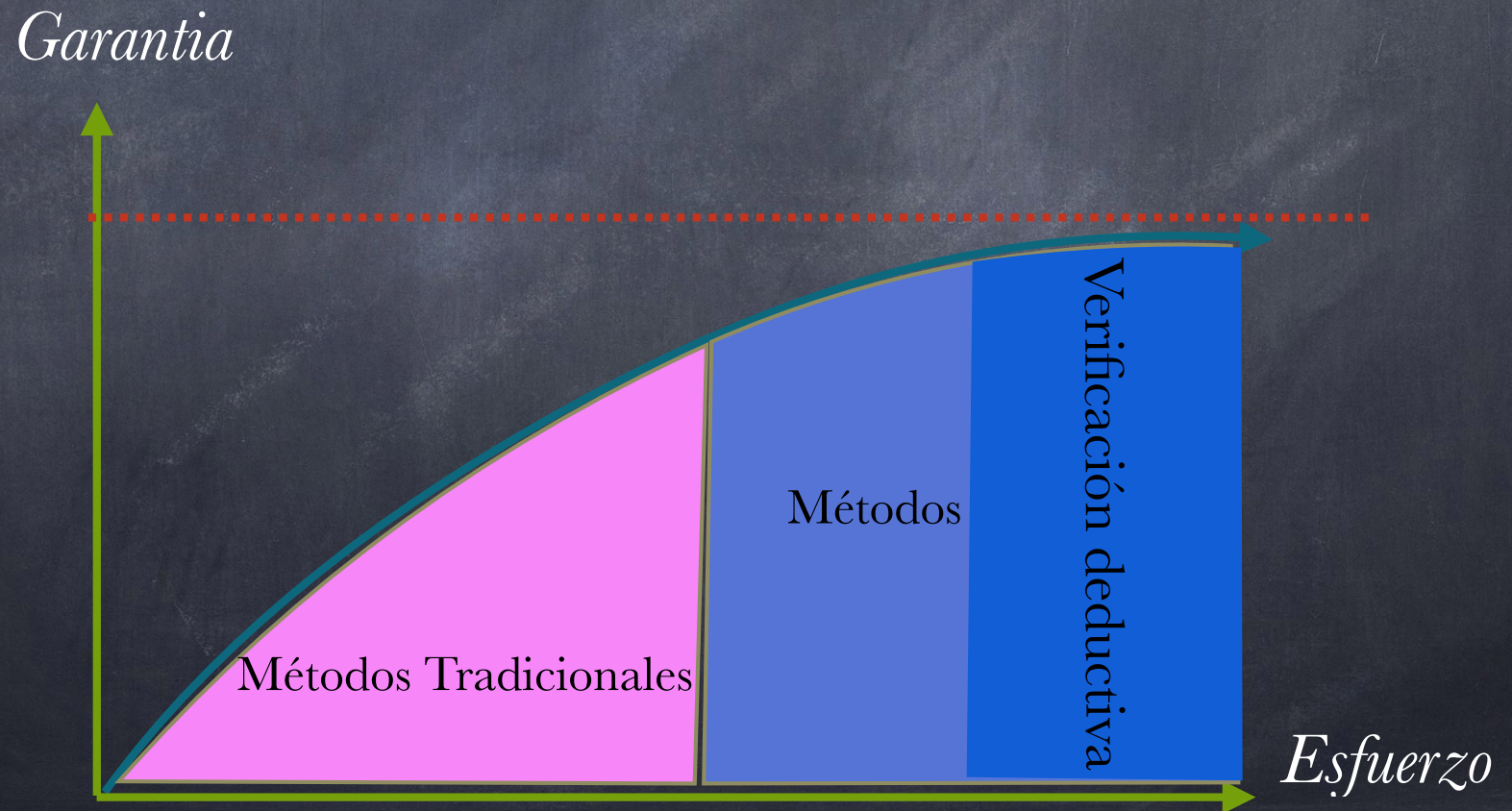
- Tradicional: No se cuenta con nada formal (excepto por el programa ejecutable).
- Formales: verificación deductiva, Model Checking

Garantía



Enfoques

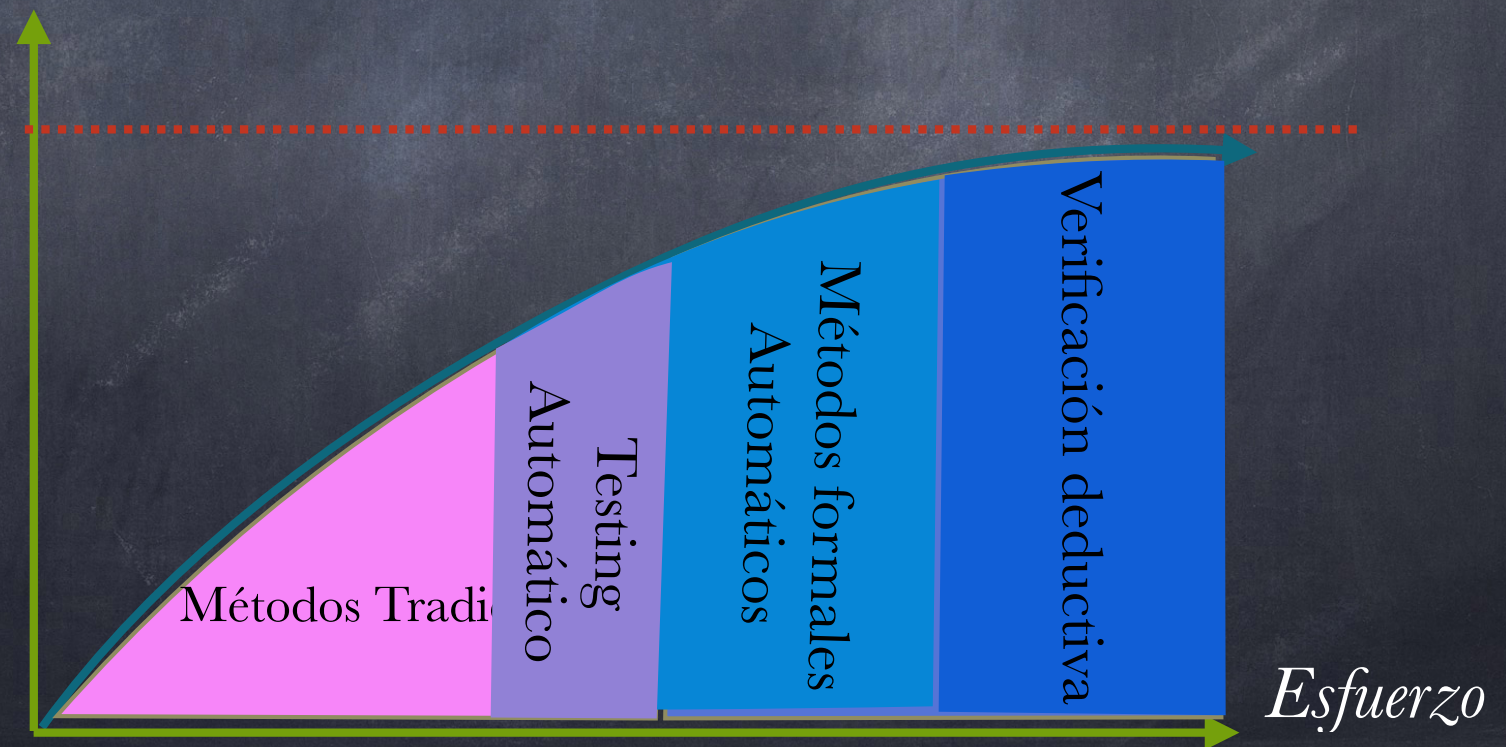
- Tradicional: No se cuenta con nada formal (excepto por el programa ejecutable).
- Formales: verificación deductiva, Model Checking



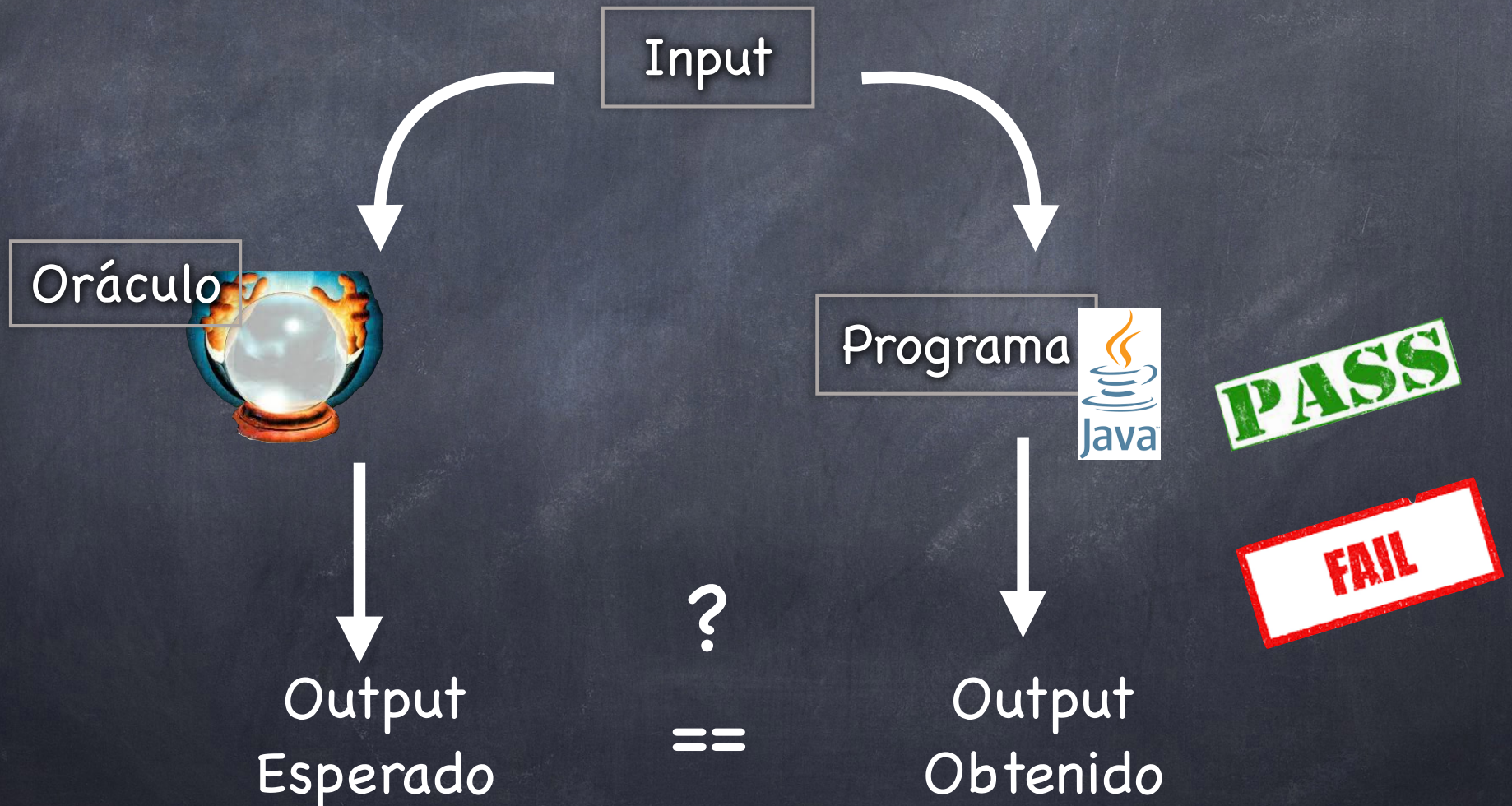
Enfoques

- Tradicional: No se cuenta con nada formal (excepto por el programa ejecutable).
- Formales: verificación deductiva, Model Checking

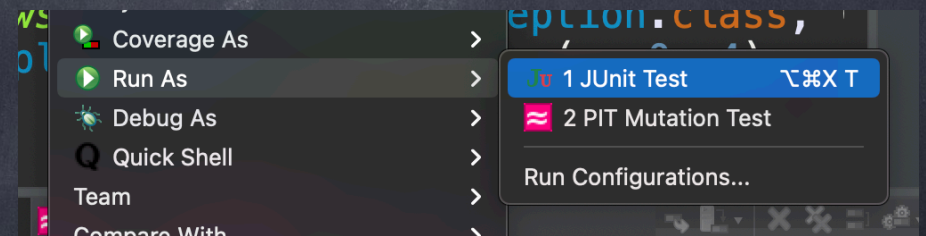
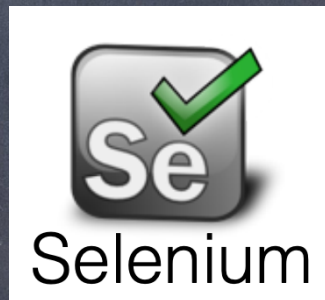
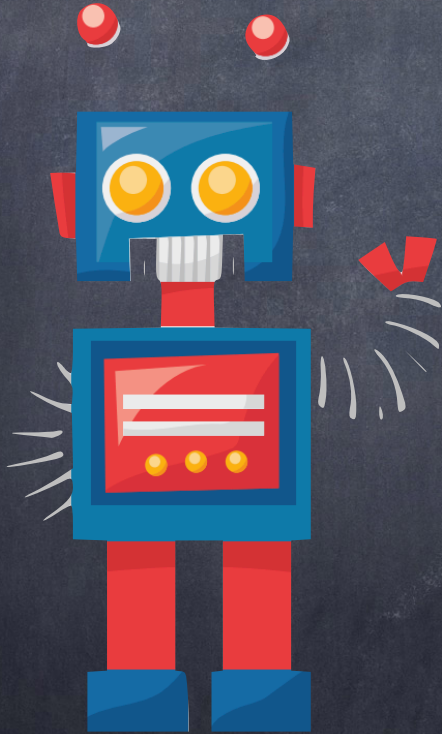
Garantía



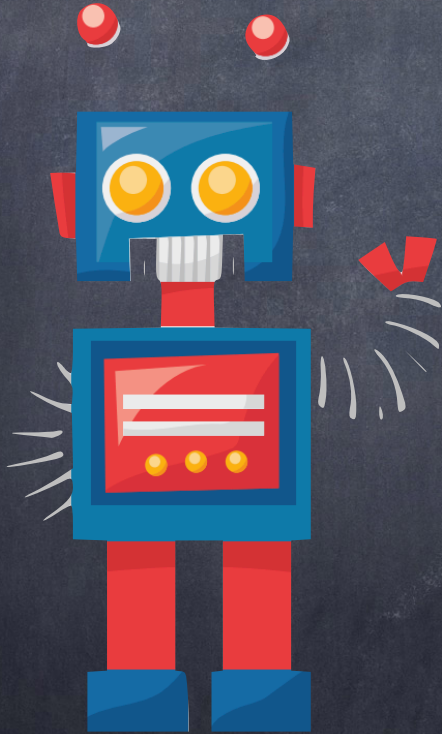
Cómo hacemos Testing?



Ejecución Automática de Casos de Test



Ejecución Automática de Casos de Test

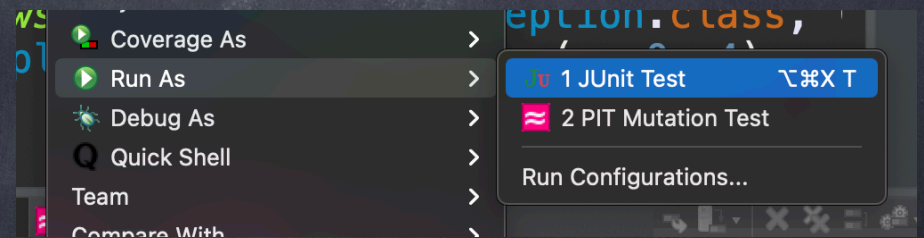


```
@Test
public void testMax() {
    int a = 1;
    int b = 3;
    int res = SimpleRoutines.max(a,b);
    assertTrue(res == 3);
}

@Test
public void largestTest() {
    Integer[] a = {1, 2, 3};

    Integer l = SimpleRoutines.largest(a);

    assertTrue(l.equals(3));
}
```



Creación Manual de Casos de Test



Creación Manual de Casos de Test



Costosa

Creación Manual de Casos de Test



Costosa

Tediosa

Creación Manual de Casos de Test



Costosa

Tediosa

Incompleta

Creación Automática de Test

Creación Automática de Test

Idea:

Aprovechar el actual poder de cómputo para

- Mejorar la eficiencia de los programadores
- Aumentar la calidad de los programas

¿Cómo?

Automatizando la creación de los Casos de Tests

Creación Automática de Test

Programa



Creación Automática de Test

Programa



Creación Automática de Test

Programa

Suite de Test



```
@Test
public void test1() {
    int a = 1;
    int b = 3;
    int res = SimpleRoutines.max(a,b);
    assertTrue(res == 3);
}

@Test
public void test2() {
    Integer[] a = {1, 2, 3};

    Integer l = SimpleRoutines.largest(a);

    assertTrue(l.equals(3));
}
```

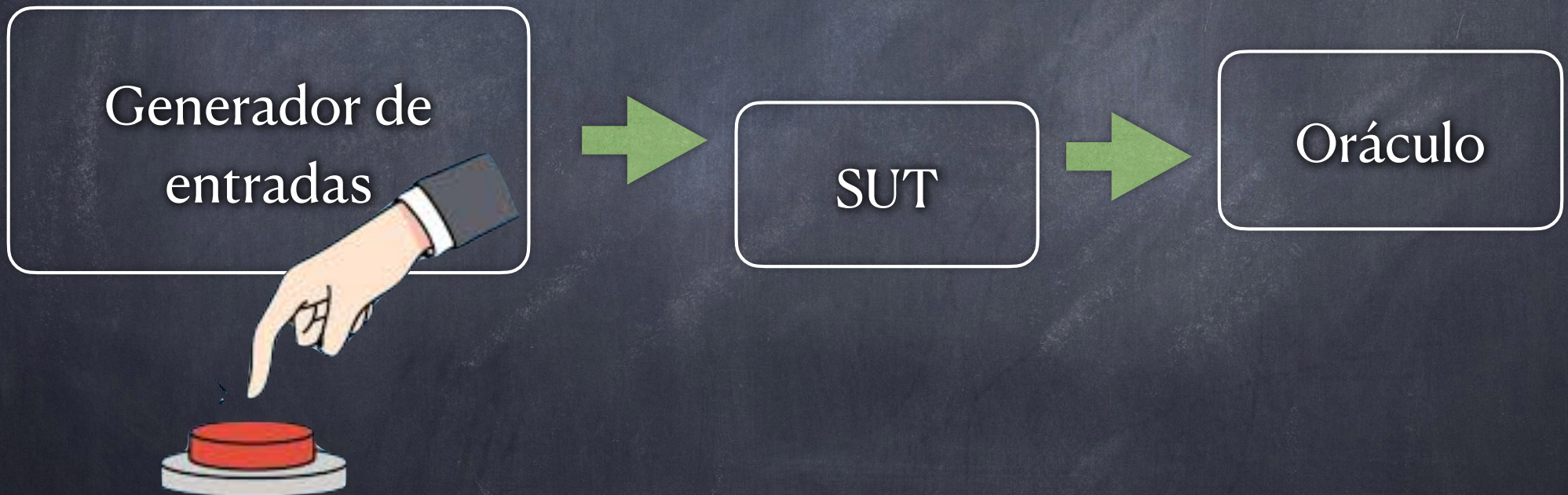

Generación Automática de Tests

La construcción de tests involucra como mínimo lo siguiente:

- la construcción de **inputs** para el SUT, que lo ejerciten en una variedad de situaciones (en general, teniendo en cuenta algún criterio de cobertura)
- la evaluación de cuál debería ser el resultado en cada caso, y la producción de los “**asserts**” correspondientes

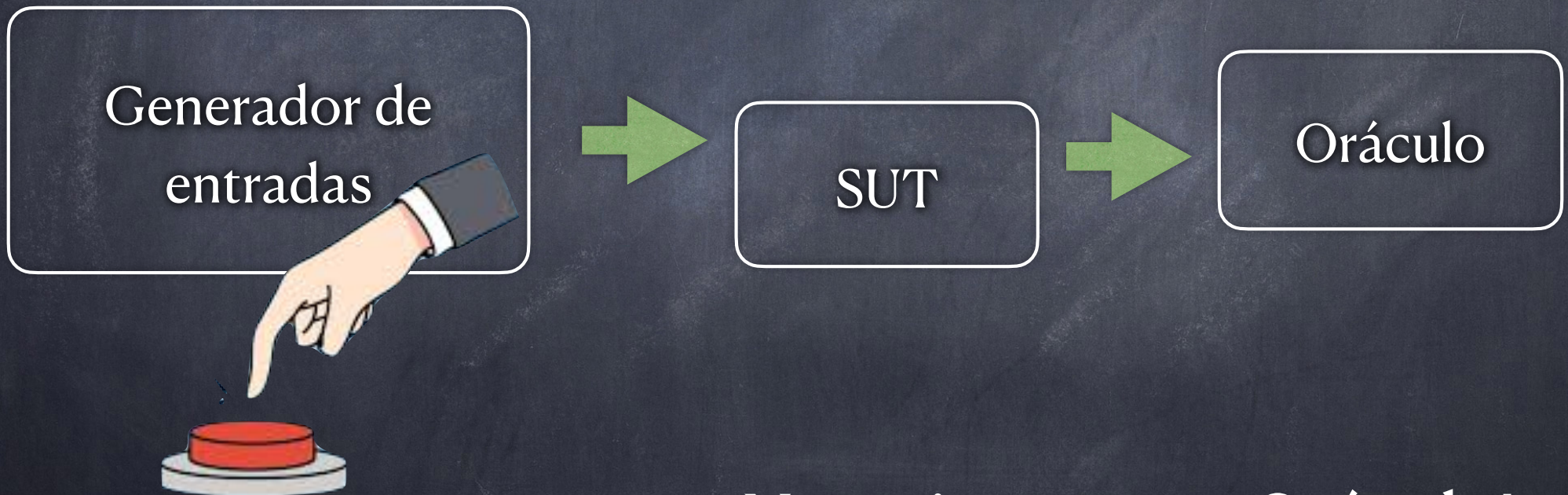
Generación Automática de Tests

Escenarios



Generación Automática de Tests

Escenarios

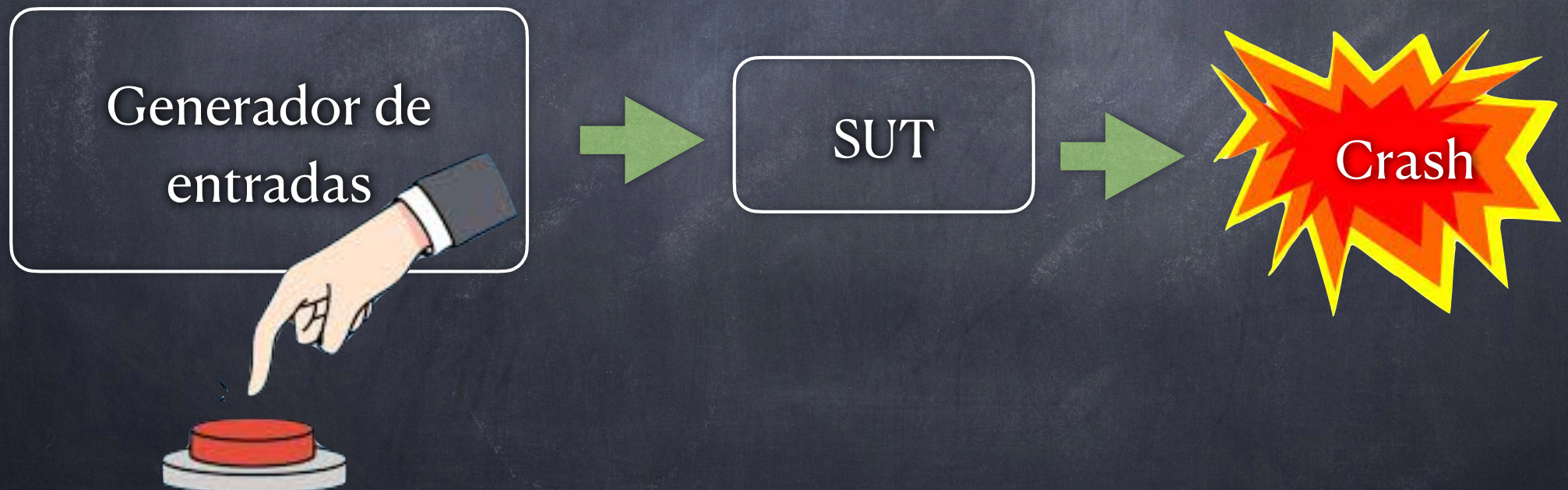


Necesitamos un Oráculo!

Generación Automática de Tests

Escenarios

A veces no contamos con un oráculo para chequear automáticamente el programa



Generación Automática de Tests

Escenarios

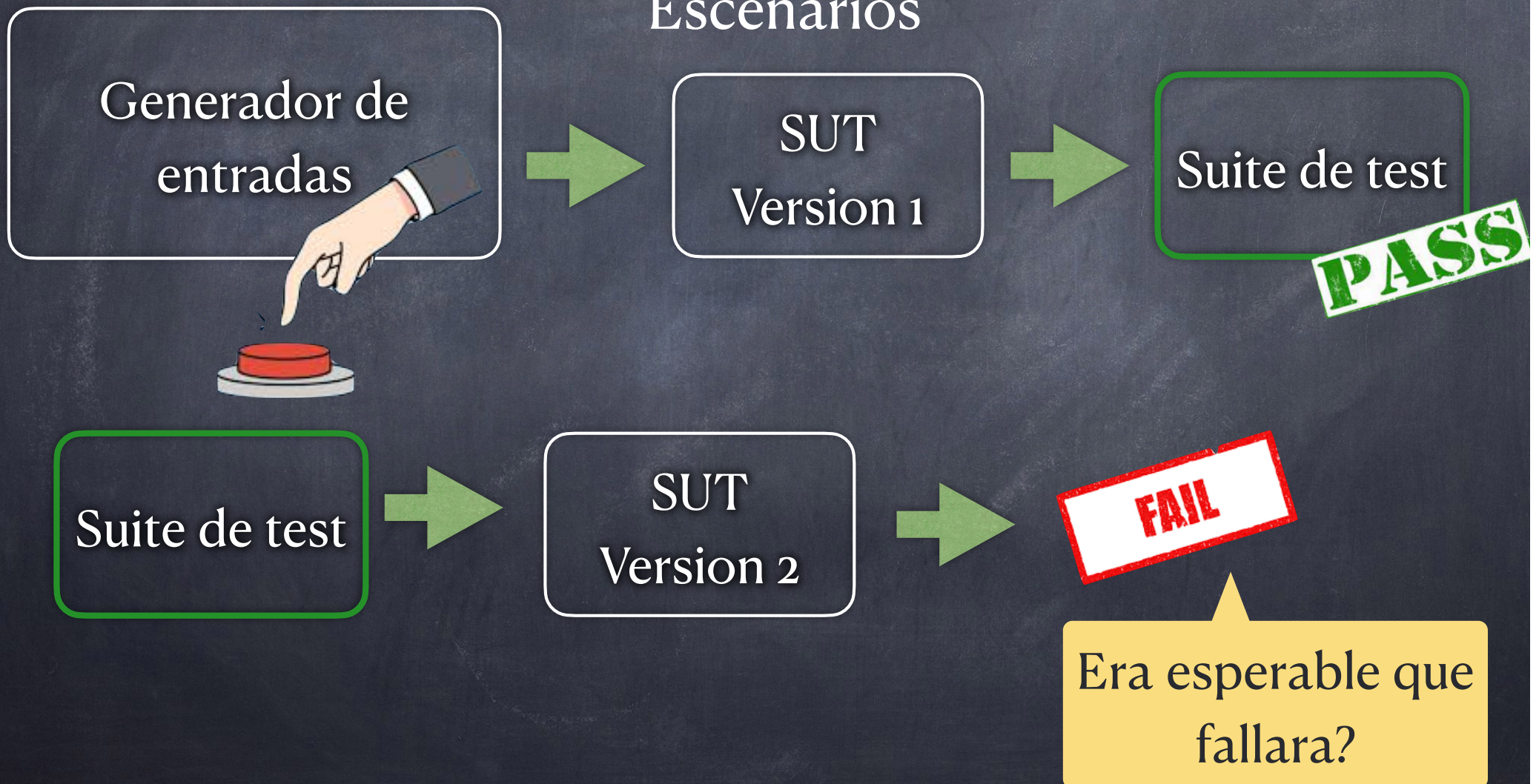
A veces no contamos con un oráculo para chequear automáticamente el programa



Testing de Robustez

Generación Automática de Tests

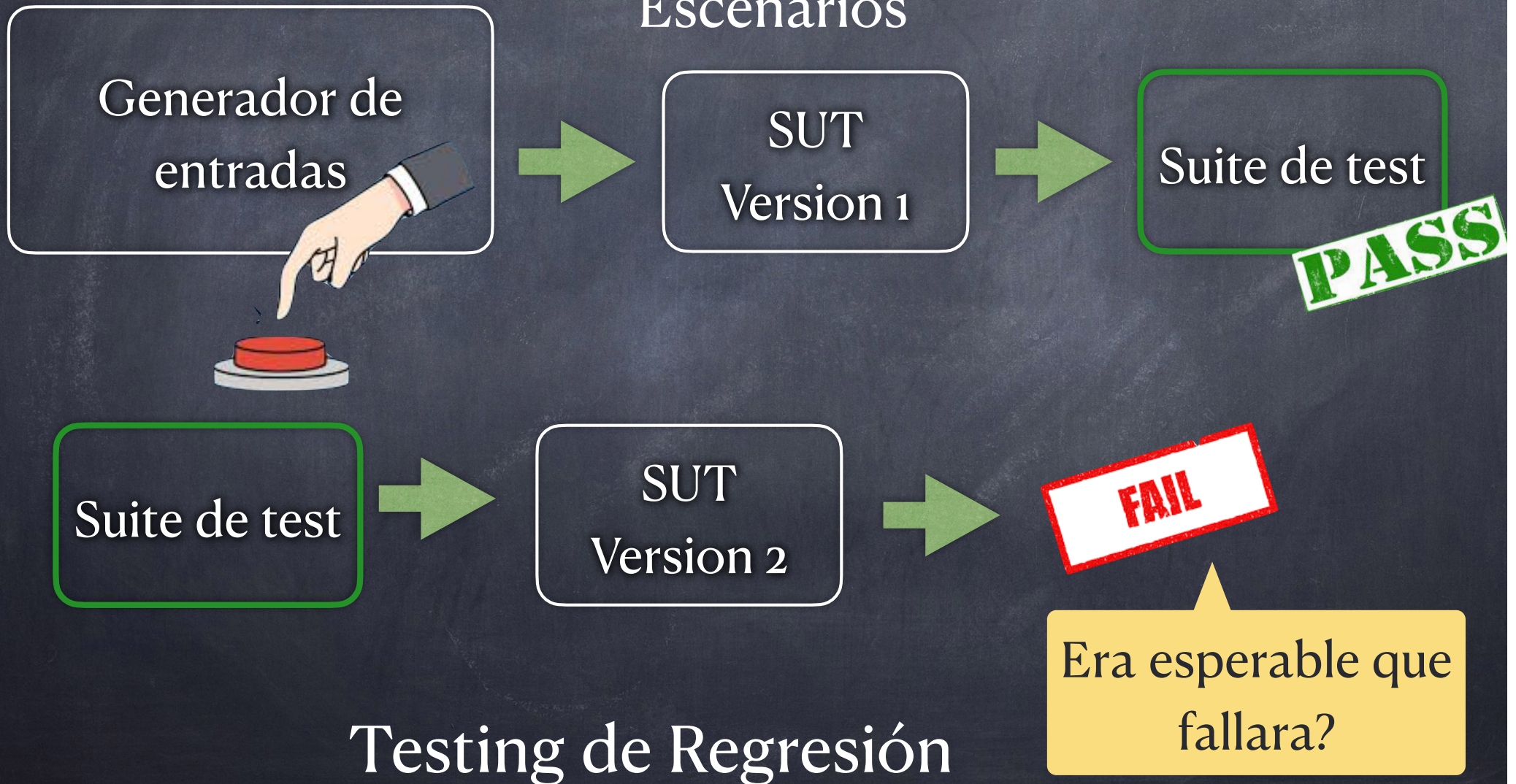
Escenarios



Generación Automática de

Tests

Escenarios



Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo **aleatoriamente** entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
 - utilización de mecanismos de generación aleatoria de valores numéricos, ej: QuickCheck, JQwik, etc

Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo **aleatoriamente** entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
- utilización de mecanismos de generación aleatoria de valores numéricos, ej: QuickCheck, JQwik, etc

```
@ParameterizedTest
@MethodSource("randomList")
void testCurrentYearParamTest(int inputDays) {
    . . .

    int result = SimpleRoutines.currentYear(inputDays);

    assertThat(result, equalTo(oracleYear));
}
```


Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo **aleatoriamente** entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
- utilización de mecanismos de generación aleatoria de valores numéricos, ej: QuickCheck, JQwik, etc

```
static IntStream randomList() {  
    java.util.Random random = new java.util.Random();  
    IntStream intStream = random.ints(1, 1000);  
    return intStream.limit(100);  
}
```

```
@ParameterizedTest  
@MethodSource("randomList")  
void testCurrentYearParamTest(int inputDays) {  
    . . .  
  
    int result = SimpleRoutines.currentYear(inputDays);  
  
    assertThat(result, equalTo(oracleYear));  
}
```


Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo **aleatoriamente** entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
- utilización de mecanismos de generación aleatoria de valores numéricos, ej: QuickCheck, JQwik, etc

```
@ParameterizedTest
@MethodSource("randomList")
void testCurrentYearParamTest(int inputDays
    . . .

    int result = SimpleRoutines.currentYear
    assertThat(result, equalTo(oracleYear))
}
```

```
static IntStream randomList() {
    java.util.Random random = new java.util.Random();
    IntStream intStream = random.ints(1, 1000);
    return intStream.limit(100);
}
```

```
@Property(tries=10000)
void testCurrentYear(@ForAll @Positive int inputDays) {

    Assume.that(inputDays>0);
    . . .

    int result = SimpleRoutines.currentYear(inputDays);
    assertThat(result, equalTo(oracleYear));
}
```


Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo **aleatoriamente** entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
- utilización de mecanismos de generación aleatoria de valores numéricos, ej: QuickCheck, JQwik, etc

Recordatorio: Necesitamos un oráculo

```
@ParameterizedTest
@MethodSource("randomList")
void testCurrentYearParamTest(int inputDays
    . . .

    int result = SimpleRoutines.currentYear
    assertThat(result, equalTo(oracleYear))
}
```

```
static IntStream randomList() {
    java.util.Random random = new java.util.Random();
    IntStream intStream = random.ints(1, 1000);
    return intStream.limit(100);
}
```

```
@Property(tries=10000)
void testCurrentYear(@ForAll @Positive int inputDays) {

    Assume.that(inputDays>0);
    . . .

    int result = SimpleRoutines.currentYear(inputDays);
    assertThat(result, equalTo(oracleYear));
}
```


Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo **aleatoriamente** entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
- utilización de mecanismos de generación aleatoria de valores numéricos, ej: QuickCheck, JQwik, etc
- Qué sucede cuando el SUT recibe entradas complejas??

Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo **aleatoriamente** entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
- utilización de mecanismos de generación aleatoria de valores numéricos, ej: QuickCheck, JQwik, etc
- Qué sucede cuando el SUT recibe entradas complejas??

```
@Property(tries = 10)
void boundedQueueProperty(@ForAll("queueGenerator") BoundedQueue queue) {
    Assume.that(!queue.isFull());
    int oldSize = queue.size();
    queue.enqueue(2);
    assertThat(queue.size(), is(oldSize+1));
}
```


Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo **aleatoriamente** entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
- utilización de mecanismos de generación aleatoria de valores numéricos, ej: QuickCheck, JQwik, etc
- Qué sucede cuando el SUT recibe entradas complejas??

Tengo que programar el generador de aleatorio!

```
@Property(tries = 10)
void boundedQueueProperty(@ForAll("queueGenerator") BoundedQueue queue) {
    Assume.that(!queue.isFull());
    int oldSize = queue.size();
    queue.enqueue(2);
    assertThat(queue.size(), is(oldSize+1));
}
```


Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo aleatoriamente entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
- utilización de mecanismos de generación aleatoria de valores numéricos, ej: JQuickCheck, JQwik, etc

Generación Aleatoria de Tests

Una forma de construir tests automáticamente es produciendo aleatoriamente entradas para los programas a testear

- simple si el SUT recibe sólo entradas de tipos simples
 - utilización de mecanismos de generación aleatoria de valores numéricos, ej: JQuickCheck, JQwik, etc
- no tan trivial si el SUT recibe entradas de tipos complejos (e.g., objetos)

Generación Aleatoria

Simple

Consideremos la siguiente técnica:

- Sea $\text{prog}(x_1: T_1, \dots, x_K: T_K)$ una rutina a testear.
- Para cada x_i hacer
 - Si T_i es un tipo básico, generar un valor v_i para asignar a x_i utilizando un generador de números aleatorios
 - Si T_i es una clase, elegir aleatoriamente entre:
 - asignar (cuando sea posible) null a x_i
 - elegir uno de los constructores sin parámetros de T_i para construir un objeto a asignar a x_i

Generación Aleatoria Simple

Ejemplo

Consideremos ahora la siguiente rutina:

```
public static Integer largest(Integer[] list) {  
    int index = 1;  
    int max = Integer.MIN_VALUE;  
    while (index <= list.length-1) {  
        if (list[index] > max) {  
            max = list[index];  
        }  
        index++;  
    }  
    return max;  
}
```


Generación Aleatoria Simple

Ejemplo

Consideremos ahora la siguiente rutina:

```
public static Integer largest(Integer[] list) {  
    int index = 1;  
    int max = Integer.MIN_VALUE;  
    while (index <= list.length-1) {  
        if (list[index] > max) {  
            max = list[index];  
        }  
        index++;  
    }  
    return max;  
}
```

```
@Test  
public void largestTest() {  
    //arrange  
  
    Integer l = SampleStaticRoutines.largest(a);  
  
    //assert  
}
```


Generación Aleatoria Simple

Ejemplo

Consideremos ahora la siguiente rutina:

```
public static Integer largest(Integer[] list) {  
    int index = 1;  
    int max = Integer.MIN_VALUE;  
    while (index <= list.length-1) {  
        if (list[index] > max) {  
            max = list[index];  
        }  
        index++;  
    }  
    return max;  
}
```

Valores posibles para a:

list = null,

list = [null, null],

list = [null, null, null, null, null]

....

```
@Test  
public void largestTest() {  
    //arrange  
  
    Integer l = SampleStaticRoutines.largest(a);  
  
    //assert  
}
```


Generación Aleatoria Simple

Ejemplo

Consideremos ahora la siguiente Clase:

```
public class BoundedQueue {  
  
    private Integer [] elems;  
    private int size;  
  
    public BoundedQueue(int c) {. . .}  
    public BoundedQueue() {. . .}  
  
    public int size() {. . .}  
    public boolean isEmpty() {. . .}  
    public void enqueue(Integer e) {. . .}  
    public Integer dequeue() {. . .}  
  
}
```


Generación Aleatoria Simple

Ejemplo

Consideremos ahora la siguiente Clase:

```
public class BoundedQueue {  
  
    private Integer [] elems;  
    private int size;  
  
    public BoundedQueue(int c) {. . .}  
    public BoundedQueue() {. . .}  
  
    public int size() {. . .}  
    public boolean isEmpty() {. . .}  
    public void enqueue(Integer e) {. . .}  
    public Integer dequeue() {. . .}  
  
}
```

```
@Test  
    public void test028() throws Throwable {  
        //arrange  
  
        boundedQueue.enqueue(a);  
  
        //asserts  
  
    }
```


Generación Aleatoria Simple

Ejemplo

Consideremos ahora la siguiente Clase:

```
public class BoundedQueue {  
  
    private Integer [] elems;  
    private int size;  
  
    public BoundedQueue(int c) {. . .}  
    public BoundedQueue() {. . .}  
  
    public int size() {. . .}  
    public boolean isEmpty() {. . .}  
    public void enqueue(Integer e) {. . .}  
    public Integer dequeue() {. . .}  
  
}
```

```
@Test  
public void test028() throws Throwable {  
    //arrange  
  
    boundedQueue.enqueue(a);  
  
    //asserts  
}
```

Valor numérico
aleatorio

Generación Aleatoria Simple

Ejemplo

Consideremos ahora la siguiente Clase:

```
public class BoundedQueue {  
  
    private Integer [] elems;  
    private int size;  
  
    public BoundedQueue(int c) {. . .}  
    public BoundedQueue() {. . .}  
  
    public int size() {. . .}  
    public boolean isEmpty() {. . .}  
    public void enqueue(Integer e) {. . .}  
    public Integer dequeue() {. . .}  
  
}
```

Algunos de los constructores
para construir “boundedQueue”

Valor numérico
aleatorio

```
@Test  
public void test028() throws Throwable {  
    //arrange  
  
    boundedQueue.enqueue(a);  
  
    //asserts  
}
```


Generación Aleatoria Simple

Ejemplo

Solo puedo construir pilas vacías de esta manera!

Consideremos ahora la siguiente Clase:

```
public class BoundedQueue {  
  
    private Integer [] elems;  
    private int size;  
  
    public BoundedQueue(int c) {. . .}  
    public BoundedQueue() {. . .}  
  
    public int size() {. . .}  
    public boolean isEmpty() {. . .}  
    public void enqueue(Integer e) {. . .}  
    public Integer dequeue() {. . .}  
  
}
```

Algunos de los constructores para construir “boundedQueue”

Valor numérico aleatorio

```
@Test  
public void test028() throws Throwable {  
    //arrange  
  
    boundedQueue.enqueue(a);  
  
    //asserts  
}
```


Generación Aleatoria Simple 2

Consideremos la siguiente técnica:

- Sea $\text{prog}(x_1: T_1, \dots, x_K: T_K)$ una rutina a testear.
 - Para cada x_i hacer
 - Si T_i es un tipo básico, generar un valor v_i para asignar a x_i utilizando un generador de números aleatorios
 - Si T_i es una clase, elegir aleatoriamente entre:
 - programar un generador aleatorio para la clase (como en las `@property`) combinando aleatoriamente invocaciones métodos de la clase que sabemos son “builders”
 - Combinar métodos de la clase aleatoriamente

Generación Aleatoria Simple 2

Debo construirlos para cada clase de objetos (no es automático!)

Consideremos la siguiente técnica:

- Sea $\text{prog}(x_1: T_1, \dots, x_K: T_K)$ una rutina a testear.
- Para cada x_i hacer
 - Si T_i es un tipo básico, generar un valor v_i para asignar a x_i utilizando un generador de números aleatorios
 - Si T_i es una clase, elegir aleatoriamente entre:
 - programar un generador aleatorio para la clase (como en las @property) combinando aleatoriamente invocaciones métodos de la clase que sabemos son “builders”
 - Combinar métodos de la clase aleatoriamente

Generación Aleatoria Simple 2

Debo construirlos para cada clase de objetos (no es automático!)

Consideremos la siguiente técnica:

- Sea $\text{prog}(x_1: T_1, \dots, x_K: T_K)$ una rutina a testear.
 - Para cada x_i hacer
 - Si T_i es un tipo básico, generar un valor v_i para asignar a x_i utilizando un generador de números aleatorios
 - Si T_i es una clase, elegir aleatoriamente entre:
 - programar un generador aleatorio para la clase (como en las @property) combinando aleatoriamente invocaciones métodos de la clase que sabemos son “builders”
- Combinar métodos de la clase aleatoriamente

Limitaciones de la Generación Aleatoria Simple

La generación aleatoria simple tiene limitaciones

- Para objetos, puede generar en general objetos muy elementales, los construibles directamente a partir de constructores simples.
- Cada generación es independiente de las anteriores, lo cual puede producir muchos tests “redundantes” e “ilegales”, que ejerciten el código de la misma manera, o violen la precondition del SUT, respectivamente.

Generación de Objetos “Complejos”

Construir entradas incrementalmente

- Nuevas entradas de test extienden a las anteriores
- En este contexto, un input es una secuencia de métodos

Ejecutar cada input creado

Usar los resultados de la ejecución para guiar la generación:

- para evitar secuencias redundantes o ilegales
- para construir secuencias que crean nuevos objetos

Ejemplo

Secuencias ya
generadas

LList API

LList()
add(int elem)
remove(int index)
contains(int elem)

```
LList l =new LList();
```

```
LList l =new LList();  
l.add(2);
```

```
LList l =new LList();  
l.add(2);  
l.add(3);
```


Ejemplo

Secuencias ya
generadas

LList API

LList()

add(int elem)

remove(int index)

contains(int elem)

remove(int index)

```
LList l =new LList();
```

```
LList l =new LList();  
l.add(2);
```

```
LList l =new LList();  
l.add(2);  
l.add(3);
```


Ejemplo

Secuencias ya
generadas

LList API

LList()
add(int elem)
remove(int index)
contains(int elem)

```
LList l = new LList();  
l.add(2);  
l.add(3);  
remove(int index)
```

```
LList l = new LList();
```

```
LList l = new LList();  
l.add(2);
```

```
LList l = new LList();  
l.add(2);  
l.add(3);
```


Ejemplo

Secuencias ya
generadas

LList API

LList()
add(int elem)
remove(int index)
contains(int elem)

```
LList l = new LList();  
l.add(2);  
l.add(3);  
l.remove(int index);
```

```
LList l = new LList();
```

```
LList l = new LList();  
l.add(2);
```

```
LList l = new LList();  
l.add(2);  
l.add(3);
```


Ejemplo

Secuencias ya
generadas

LList API

LList()
add(int elem)
remove(int index)
contains(int elem)

```
LList l = new LList();  
l.add(2);  
l.add(3);  
l.remove(0);
```

```
LList l = new LList();
```

```
LList l = new LList();  
l.add(2);
```

```
LList l = new LList();  
l.add(2);  
l.add(3);
```


Ejemplo

Secuencias ya
generadas

LList API

LList()
add(int elem)
remove(int index)
contains(int elem)

```
LList l = new LList();  
l.add(2);  
l.add(3);  
l.remove(0);
```

```
LList l = new LList();
```

```
LList l = new LList();  
l.add(2);
```

```
LList l = new LList();  
l.add(2);  
l.add(3);
```

```
LList l = new LList();  
l.add(2);  
l.add(3);  
l.remove(0);
```


Generación aleatoria

Problemas

En la generación aleatoria de tests, es posible caer en las siguientes situaciones:

```
Set s = new HashSet();  
s.add("hi");  
assertTrue(s.size()==1);
```



```
Set s = new HashSet();  
s.add("hi");  
s.isEmpty();  
assertTrue(s.size()==1);
```

```
Date d = new Date(2006,2,14);  
assertTrue( ... );
```



```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // pre:argument>0  
assertTrue(d.equals(d));
```

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // lanza excepción  
d.setDay(5);  
assertTrue( ... );
```


Generación aleatoria

Problemas

En la generación aleatoria de tests, es posible caer en las siguientes situaciones:



```
Set s = new HashSet();  
s.add("hi");  
assertTrue(s.size()==1);
```

Redundante

```
Set s = new HashSet();  
s.add("hi");  
s.isEmpty();  
assertTrue(s.size()==1);
```



```
Date d = new Date(2006,2,14);  
assertTrue( ... );
```

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // pre:argument>0  
assertTrue(d.equals(d));
```

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // lanza excepción  
d.setDay(5);  
assertTrue( ... );
```


Generación aleatoria

Problemas

En la generación aleatoria de tests, es posible caer en las siguientes situaciones:



```
Set s = new HashSet();  
s.add("hi");  
assertTrue(s.size()==1);
```

Redundante

```
Set s = new HashSet();  
s.add("hi");  
s.isEmpty();  
assertTrue(s.size()==1);
```

Descartar



```
Date d = new Date(2006,2,14);  
assertTrue( ... );
```

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // pre:argument>0  
assertTrue(d.equals(d));
```

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // lanza excepción  
d.setDay(5);  
assertTrue( ... );
```


Generación aleatoria

Problemas

En la generación aleatoria de tests, es posible caer en las siguientes situaciones:



```
Set s = new HashSet();  
s.add("hi");  
assertTrue(s.size()==1);
```

Redundante

```
Set s = new HashSet();  
s.add("hi");  
s.isEmpty();  
assertTrue(s.size()==1);
```

Descartar



```
Date d = new Date(2006,2,14);  
assertTrue( ... );
```

Illegal

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // pre:argument>0  
assertTrue(d.equals(d));
```

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // lanza excepción  
d.setDay(5);  
assertTrue( ... );
```


Generación aleatoria

Problemas

En la generación aleatoria de tests, es posible caer en las siguientes situaciones:



```
Set s = new HashSet();  
s.add("hi");  
assertTrue(s.size()==1);
```

Redundante

```
Set s = new HashSet();  
s.add("hi");  
s.isEmpty();  
assertTrue(s.size()==1);
```

Descartar



```
Date d = new Date(2006,2,14);  
assertTrue( ... );
```

Illegal

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // pre:argument>0  
assertTrue(d.equals(d));
```

Descartar

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // lanza excepción  
d.setDay(5);  
assertTrue( ... );
```


Generación aleatoria

Problemas

En la generación aleatoria de tests, es posible caer en las siguientes situaciones:



```
Set s = new HashSet();  
s.add("hi");  
assertTrue(s.size()==1);
```

Redundante

```
Set s = new HashSet();  
s.add("hi");  
s.isEmpty();  
assertTrue(s.size()==1);
```

Descartar



```
Date d = new Date(2006,2,14);  
assertTrue( ... );
```

Illegal

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // pre:argument>0  
assertTrue(d.equals(d));
```

Descartar

Illegal

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // lanza excepción  
d.setDay(5);  
assertTrue( ... );
```


Generación aleatoria

Problemas

En la generación aleatoria de tests, es posible caer en las siguientes situaciones:



```
Set s = new HashSet();  
s.add("hi");  
assertTrue(s.size()==1);
```

Redundante

```
Set s = new HashSet();  
s.add("hi");  
s.isEmpty();  
assertTrue(s.size()==1);
```

Descartar



```
Date d = new Date(2006,2,14);  
assertTrue( ... );
```

Illegal

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // pre:argument>0  
assertTrue(d.equals(d));
```

Descartar

Illegal

```
Date d = new Date(2006,2,14);  
d.setMonth(-1); // lanza excepción  
d.setDay(5);  
assertTrue( ... );
```

No generar

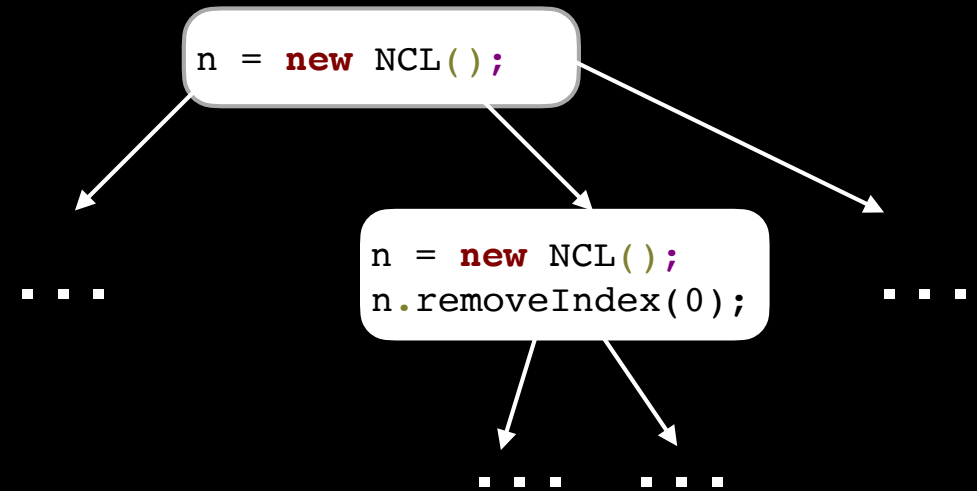
Técnica: Generación Aleatoria Guiada por Feedback

La generación aleatoria de tests **guiada por feedback** complementa el proceso de generación descrito anteriormente, mediante la **construcción incremental** de entradas

- Tan pronto como se produzca una secuencia de generación, se ejecuta la misma
- Si la secuencia es **válida**, se producen nuevas extendiendo a ésta y otras anteriores

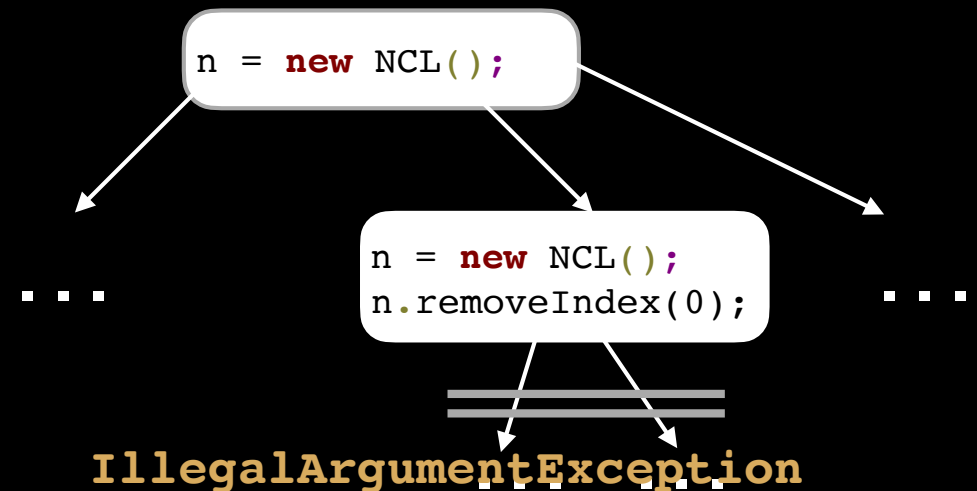
Técnica: Generación Aleatoria Guiada por Feedback

- Ejecuta secuencia de test para observar su comportamiento
- No se extienden secuencias que violan precondiciones (e.g. `IllegalArgumentException`)
- Report bugs found (e.g. `NullPointerException`)



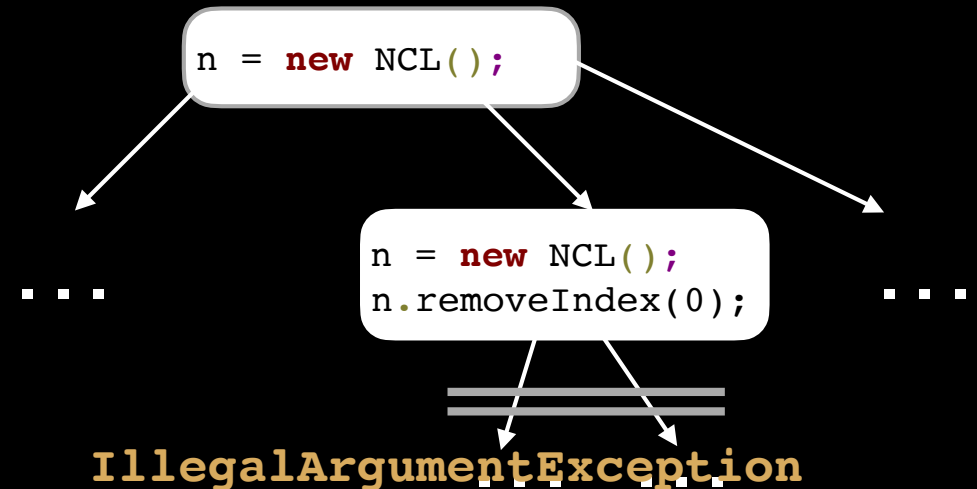
Técnica: Generación Aleatoria Guiada por Feedback

- Executa secuencia de test para observar su comportamiento
- No se extienden secuencias que violan precondiciones (e.g. `IllegalArgumentException`)
- Report bugs found (e.g. `NullPointerException`)



Técnica: Generación Aleatoria Guiada por Feedback

- Executa secuencia de test para observar su comportamiento
- No se extienden secuencias que violan precondiciones (e.g. `IllegalArgumentException`)
- Report bugs found (e.g. `NullPointerException`)



```
public Object removeIndex(int index) {  
    // Check precondition  
    if(index < 0 || index >= size)  
        throws new IllegalArgumentException();  
    . . .  
}
```

Defensive programming: Code must fail when a precondition is violated

Técnica: Entradas

- Clase bajo tests
- Límite de tiempo o cantidad máxima de test
- Oráculo (Contratos)
 - Sobre el equals: reflexividad, simetria, transitividad, etc.
 - Equals and hashCode son consistentes: Objetos que son iguales deben retornar el mismo hashCode
 - otros
 - Contratos provistos por el usuario: Invariante de Representación

Técnica: Salid

Regression tests

- Conjuntos de test con comportamiento normal (test que pasan)
- Ejemplo test que viola contratos (Oráculos implícitos en POO)

Error tests

```
practico1.point.Point point1 = new practico1.point.Point(0.0f, 0.0f);
practico1.point.Point point2 = new practico1.point.Point(0.0f, 0.0f);
java.lang.Double d33 = point2.distanceTo(point1);

// Checks the contract: equals-hashcode on point7 and point19
assertTrue("Contract failed: equals-hashcode on point7 and
    point19", point1.equals(point2) ? point1.hashCode() == point2.hashCode() : true);
```

$p1.equals(p2) \implies p1.hashCode() == p2.hashCode()$

Técnica: Salidas

- Conjunto de tests que violan contratos

```
@Test
public void test017() throws Throwable {
    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();
    java.lang.Object obj8 = null;
    listaSobreArreglos0.insertar(obj8);
    java.lang.Object obj11 = listaSobreArreglos0.obtener(0);
    // during test generation this statement threw an exception of
    // type java.lang.NullPointerException in error
    java.lang.String str12 = listaSobreArreglos0.toString();
}
```

Falla cuando se ejecuta

Técnica: Salidas

- Conjunto de tests que violan contratos

```
@Test
public void test017() throws Throwable {
    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();
    java.lang.Object obj8 = null;
    listaSobreArreglos0.insertar(obj8);
    java.lang.Object obj11 = listaSobreArreglos0.obtener(0);
    // during test generation this statement threw an exception of
    // type java.lang.NullPointerException in error
    java.lang.String str12 = listaSobreArreglos0.toString();
}
```

Testing de Robustez

Falla cuando se ejecuta

Generación aleatoria guiada por feedback

1. secuencias semillas (de tipo primitivo)

`secuencias_previas = {int =0, boolean b = false}`

2. Hacer hasta que el límite de tiempo (o cantidad de test) expira:

A. Crear una nueva secuencia

I. Seleccionar aleatoriamente una invocación a método $m(T_1...T_k)/T_{ret}$

II. Para cada parámetro de entrada de tipo T_i , seleccionar aleatoriamente una secuencia S_i de `secuencias_previas` que construye un objeto v_i de tipo T_i

III. Crear una nueva secuencia

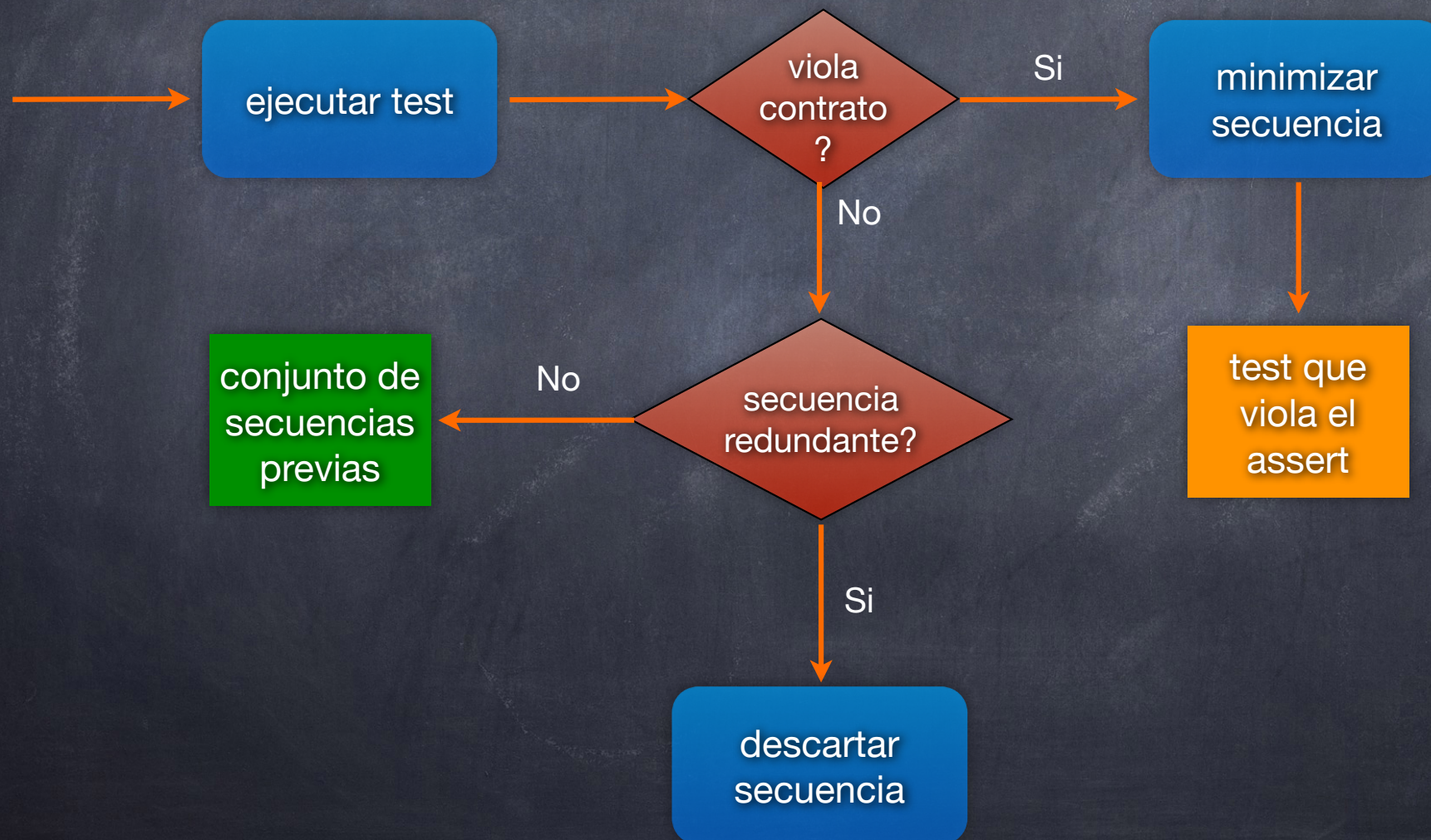
$S_{new} = S_1; S_2 ; .. ; S_k ; T_{ret} \quad v_{new} = m(v_1, v_2, ..., v_k);$

IV. Si S_{new} fue creada previamente (sintácticamente), ir a I

B. Clasificar la nueva secuencia S_{new}

Puede descartar, almacenar el caso de test, o agregar a `secuencias_previas`

Clasificación de Tests generados



Cómo Determinar si una Secuencia es Redundante

Para determinar si una secuencia es **redundante**:

- Durante la generación se mantiene el conjunto de **todos los objetos creados**.
- Cuando se produce una nueva secuencia válida, se compara con los objetos creados previamente
 - si durante su ejecución no genera nuevos objetos (usando el **equals** para comparar), entonces la secuencia generada se descarta (redundante)
 - si el input no fue creado previamente, se almacena la secuencia, y **se guarda el nuevo input generado en el conjunto de objetos**

Randoop

Randoop es una herramienta para la generación de tests unitarios basada en generación aleatoria

- incorpora las técnicas de generación guiadas por feedback descriptas para:
 - intentar eliminar casos de test inválidos
 - intentar no producir casos de test redundantes

Existen versiones de Randoop tanto para Java como para .NET

Randoop

Recibe como entrada:

- un conjunto de clases (SUT)
- un tiempo límite de generación/cantidad máxima de test

Randoop produce como salida:

- suite de tests que violan “contratos” básicos generados automáticamente,
- suite de test que son reconocidos como tests válidos (test de regresión)

Aserciones Generadas Automáticamente

Para determinar si un test corresponde a comportamiento correcto o no, Randoop genera **aserciones básicas** que, independientemente del contexto, siempre deberían ser válidos.

- Contratos sobre equals(), hashCode(), toString(), ...
Ej: **assertTrue(o.equals(o))**.
- Ver más en https://randoop.github.io/randoop/manual/#kinds_of_errors

Además, Randoop genera aserciones que capturan el comportamiento de la implementación actual para **testing de regresión**

Aserciones Generadas Automáticamente

Aserciones para un test con comportamiento normal:

```
@Test
public void test034() throws Throwable {

    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();
    java.lang.Object obj9 = null;
    boolean b10 = listaSobreArreglos0.equals(obj9);
    java.lang.Object obj17 = null;
    listaSobreArreglos0.insertar(0, obj17);
    boolean b2 = listaSobreArreglos0.esVacía();

    // Regression assertion (captures the current behavior of the code)
    assertTrue(b2 == false);
    // Regression assertion (captures the current behavior of the code)
    assertTrue(b10 == false);

}
```

Randoop usa métodos observadores para capturar el estado de los objetos

Aserciones Generadas Automáticamente

Aserciones para un test con comportamiento normal:

```
@Test
public void test034() throws Throwable {

    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();
    java.lang.Object obj9 = null;
    boolean b10 = listaSobreArreglos0.equals(obj9);
    java.lang.Object obj17 = null;
    listaSobreArreglos0.insertar(0, obj17);
    boolean b2 = listaSobreArreglos0.esVacía();

    // Regression assertion (captures the current behavior of the code)
    assertTrue(b2 == false);
    // Regression assertion (captures the current behavior of the code)
    assertTrue(b10 == false);
}
```

Randoop usa métodos observadores para capturar el estado de los objetos

Aserciones Generadas Automáticamente

Aserciones para un test con comportamiento normal:

```
@Test
public void test034() throws Throwable {

    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();
    java.lang.Object obj9 = null;
    boolean b10 = listaSobreArreglos0.equals(obj9);
    java.lang.Object obj17 = null;
    listaSobreArreglos0.insertar(0, obj17);
    boolean b2 = listaSobreArreglos0.esVacía();

    // Regression assertion (captures the current behavior of the code)
    assertTrue(b2 == false);
    // Regression assertion (captures the current behavior of the code)
    assertTrue(b10 == false);

}
```

Randoop usa métodos observadores para capturar el estado de los objetos

Aserciones Generadas Automáticamente

Aserciones para un test con comportamiento normal:

```
@Test
public void test034() throws Throwable {

    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();
    java.lang.Object obj9 = null;
    boolean b10 = listaSobreArreglos0.equals(obj9);
    java.lang.Object obj17 = null;
    listaSobreArreglos0.insertar(0, obj17);
    boolean b2 = listaSobreArreglos0.esVacía();

    // Regression assertion (captures the current behavior of the code)
    assertTrue(b2 == false);
    // Regression assertion (captures the current behavior of the code)
    assertTrue(b10 == false);

}
```

Aserciones de
regresión

Randoop usa métodos observadores para capturar el estado de los objetos

Aserciones Generadas Automáticamente

Test negativos:

```
public void test003() throws Throwable {  
    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();  
    // The following exception was thrown during execution in test generation  
    try {  
        listaSobreArreglos0.eliminar(0);  
        Assert.fail("Expected exception of type java.lang.IndexOutOfBoundsException");  
    } catch (java.lang.IndexOutOfBoundsException e) {  
        // Expected exception.  
    }  
}
```


Aserciones Generadas Automáticamente

Test negativos:

```
public void test003() throws Throwable {  
    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();  
    // The following exception was thrown during execution in test generation  
    try {  
        listaSobreArreglos0.eliminar(0);  
        Assert.fail("Expected exception of type java.lang.IndexOutOfBoundsException");  
    } catch (java.lang.IndexOutOfBoundsException e) {  
        // Expected exception.  
    }  
}
```


Aserciones Generadas Automáticamente

Test negativos:

```
public void test003() throws Throwable {  
    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();  
    // The following exception was thrown during execution in test generation  
    try {  
        listaSobreArreglos0.eliminar(0);  
        Assert.fail("Expected exception of type java.lang.IndexOutOfBoundsException");  
    } catch (java.lang.IndexOutOfBoundsException e) {  
        // Expected exception.  
    }  
}
```

Aserciones de
regresión

Testing de Regresión

- Randoop genera test de regresión los cuales capturan el comportamiento actual del software.
- Pueden ser usados para verificar que algunos componentes del software mantienen su comportamiento con respecto a versiones anteriores

Aserciones Generadas Automáticamente

Aserciones que violan contratos:

```
@Test
public void test001() throws Throwable {
    demo1.listas.ListaSobreArreglos listaSobreArreglos0 = new demo1.listas.ListaSobreArreglos();
    listaSobreArreglos0.insertar((java.lang.Object)(byte)100);
    java.lang.Object obj3 = null;
    listaSobreArreglos0.insertar(obj3);
    // during test generation this statement threw an exception of type java.lang.NullPointerException i
    java.lang.String str5 = listaSobreArreglos0.toString();
}
```

```
@Test
public void test002() throws Throwable {
    demo1.listas.ListaSobreArreglos listaSobreArreglos0 = new demo1.listas.ListaSobreArreglos();
    listaSobreArreglos0.insertar((java.lang.Object)'a');
    listaSobreArreglos0.eliminar(0);

    // Check representation invariant.
    org.junit.Assert.assertTrue("Representation invariant failed: Check rep invariant (method repOk)
                                for listaSobreArreglos0", listaSobreArreglos0.repOk());
}
```


Aserciones Generadas Automáticamente

Aserciones que violan contratos:

```
@Test
public void test001() throws Throwable {
    demo1.listas.ListaSobreArreglos listaSobreArreglos0 = new demo1.listas.ListaSobreArreglos();
    listaSobreArreglos0.insertar((java.lang.Object)(byte)100);
    java.lang.Object obj3 = null;
    listaSobreArreglos0.insertar(obj3);
    // during test generation this statement threw an exception of type java.lang.NullPointerException i
    java.lang.String str5 = listaSobreArreglos0.toString();
}
```

```
@Test
public void test002() throws Throwable {
    demo1.listas.ListaSobreArreglos listaSobreArreglos0 = new demo1.listas.ListaSobreArreglos();
    listaSobreArreglos0.insertar((java.lang.Object)'a');
    listaSobreArreglos0.eliminar(0);

    // Check representation invariant.
    org.junit.Assert.assertTrue("Representation invariant failed: Check rep invariant (method repOk)
                                for listaSobreArreglos0", listaSobreArreglos0.repOk());
}
```


Aserciones Generadas Automáticamente

Aserciones que violan contratos:

```
@Test
public void test001() throws Throwable {
    demo1.listas.ListaSobreArreglos listaSobreArreglos0 = new demo1.listas.ListaSobreArreglos();
    listaSobreArreglos0.insertar((java.lang.Object)(byte)100);
    java.lang.Object obj3 = null;
    listaSobreArreglos0.insertar(obj3);
    // during test generation this statement threw an exception of type java.lang.NullPointerException i
    java.lang.String str5 = listaSobreArreglos0.toString();
}
```

```
@Test
public void test002() throws Throwable {
    demo1.listas.ListaSobreArreglos listaSobreArreglos0 = new demo1.listas.ListaSobreArreglos();
    listaSobreArreglos0.insertar((java.lang.Object)'a');
    listaSobreArreglos0.eliminar(0);

    // Check representation invariant.
    org.junit.Assert.assertTrue("Representation invariant failed: Check rep invariant (method repOk)
                                for listaSobreArreglos0", listaSobreArreglos0.repOk());
}
```


Randoop e Invariantes de Representación

Además del chequeo de aserciones básicas, Randoop también puede utilizar una especificación provista por el usuario por ejemplo, en forma de invariante de clase.

Recordemos: Las invariantes de rep. establecen propiedades que deben mantenerse a lo largo de la ejecución de los objetos de la clase.

- todos los constructores deben establecer, al finalizar, la propiedad
- todos los métodos de la clase deben preservar la propiedad

Randoop e Invariantes de Representación

Además del chequeo de aserciones básicas, Randoop también puede utilizar una especificación provista por el usuario por ejemplo, en forma de invariante de clase.

Oráculo

Recordemos: Las invariantes de rep. establecen propiedades que deben mantenerse a lo largo de la ejecución de los objetos de la clase.

- todos los constructores deben establecer, al finalizar, la propiedad
- todos los métodos de la clase deben preservar la propiedad

Randoop e Invariantes de Representación

Ejemplo

```
import randoop.*;
```

```
public class ListaSobreArreglos implements Lista {  
    private static final int MAX_LIST = 10;  
    private Object items[];  
    private int numItems;  
    ...  
}
```

```
@CheckRep
```

```
public boolean repOk() {  
    return (0<=numItems && numItems<=MAX_LIST);  
}  
...  
}
```

Habilitar chequeo
de Invariante de
representación

Randoop e Invariantes de Representación

Ejemplo

```
@Test
public void test001() throws Throwable {

    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();
    int i1 = listaSobreArreglos0.longitud();
    boolean b2 = listaSobreArreglos0.esVacía();
    listaSobreArreglos0.insertar((java.lang.Object)10.0f);

    // Check representation invariant.
    assertTrue("Representation invariant failed: Check rep invariant (method repOk) for
               listaSobreArreglos0", listaSobreArreglos0.repOk());
}
```


Algunas Opciones de Configuración

- Evitar uso del Null

```
@Test
public void test034() throws Throwable {

    ListaSobreArreglos listaSobreArreglos0 = new ListaSobreArreglos();
    listaSobreArreglos0.insertar(0, null);
    . . .
}
```

- Omitir métodos de la generación
- Selección de cuáles test escribir como salida
 - Error test
 - Regresión test
 - Test sin aserciones
 - Test que “ejercitan” ciertas clases dadas
 - ...

Algunas Opciones de Configuración

- Clasificación de test
 - Qué tipo de excepciones serán consideradas `errorTest`
- Valores primitivos provistos por el usuario
 - De otra manera, se utilizan algunos valores por defecto además de los valores retornados por invocaciones a métodos.
 - Randoop **promete** incluir en la generación valores extraídos del código fuente

Algunas Opciones de Configuración

- Clasificación de test
 - Qué tipo de excepciones serán consideradas `errorTest`
- Valores primitivos provistos por el usuario
 - De otra manera, se utilizan algunos valores por defecto además de los valores retornados por invocaciones a métodos.
 - Randoop **promete** incluir en la generación valores extraídos del código fuente

```
int Complicated(int x) {  
    if (x == 3.141592){  
        //do something  
    }else{  
        //do something else  
    }  
}
```


Algunas Opciones de Configuración

- Clasificación de test
 - Qué tipo de excepciones serán consideradas `errorTest`
- Valores primitivos provistos por el usuario
 - De otra manera, se utilizan algunos valores por defecto además de los valores retornados por invocaciones a métodos.
 - Randoop **promete** incluir en la generación valores extraídos del código fuente

Difícil de ejercitar sin valores extras

```
int Complicated(int x) {  
    if (x == 3.141592){  
        //do something  
    }else{  
        //do something else  
    }  
}
```


Algunas Opciones de Configuración

- Selección Random no equi-probable:
 - Favorecer secuencias pequeñas
 - Favorecer métodos que han sido menos cubiertos
 - Usar constantes encontradas en el código
- Control de aleatoriedad:
 - Cambiar la semilla en el proceso de generación
- Comportamiento frente a tests no confiables (flaky)
 - Qué hacer si Randoop genera flaky test: Terminar la ejecución, descartar el test o escribir como salida
- Determinar “replacement methods” (**MockS/StubS**) para métodos (no determinísticos en general)

Flaky tests

- Un test es “flaky” o no confiable si el mismo arroja diferentes resultados en diferentes ejecuciones.
 - Producidos en general por métodos con efectos colaterales o métodos no determinísticos

Flaky tests

- Un test es “flaky” o no confiable si el mismo arroja diferentes resultados en diferentes ejecuciones.
 - Producidos en general por métodos con efectos colaterales o métodos no determinísticos

métodos que obtienen la fecha del sistema para realizar algún cálculo, uso de valores random, etc

Efectividad de Generación Aleatoria Guiada por Feedback

Ha resultado ser muy efectiva para descubrir bugs

	LOC	Classes
JDK (2 libraries) (java.util, javax.xml)	53K	272
Apache commons (5 libraries) (logging, primitives, chain jelly, math, collections)	114K	974
.Net framework (5 libraries)	582K	3330

Efectividad de Generación Aleatoria Guiada por Feedback

Ha resultado ser muy efectiva para descubrir bugs

	test cases output	error-revealing tests cases	distinct errors
JDK	32	29	8
Apache commons	187	29	6
.Net framework	192	192	192
Total	411	250	206

Otras Herramientas basadas en Generación Aleatoria

Existen otras herramientas de generación automática de tests basadas en generación aleatoria.

- AutoTest: es la herramienta de generación automática de tests para Eiffel, integrada a EiffelStudio
- QuickCheck: es una herramienta de generación aleatoria de tests para Haskell
- Monkey para Android: Generación aleatoria de eventos del usuario.

Randoop y bibliografía recomendada

- <https://randoop.github.io/randoop/manual/index.html>
- <http://people.csail.mit.edu/cpacheco/publications/randoopjava.pdf>
- <https://homes.cs.washington.edu/~mernst/pubs/feedback-testgen-icse2007.pdf>
- <https://randoop.github.io/randoop/publications.html>